

Advanced OOP in Python

FOR PRESENTER USE ONLY

Overview

This lesson uses a delivery-prediction scenario to teach core advanced OOP concepts: inheritance, polymorphism, method types, class attributes, properties, composition and context managers, with practical exercises.

There is also optional stretch content on abstract classes, mixins and MRO.

Learning objectives

- Use inheritance, overriding, and polymorphism to design reusable Python classes.
- Distinguish between instance methods, class methods, and static methods.
- Explain class attributes, instance attributes, and Python access conventions.
- Use properties to validate object state.
- Use composition to connect workflow components.
- Use context managers for predictable setup and cleanup.

FOR PRESENTER USE ONLY



Suggested Agenda

Time	Activity
0:00–0:10	**Welcome + Introduction**
0:10–0:30	**Inheritance, overriding, and polymorphism**
0:30–0:45	**DataSource code-along and Now You Try**
0:45–1:05	**Method types and class attributes**
1:05–1:20	**Experiment tracker Now You Try**
1:20–1:40	**Access conventions, properties, and composition**
1:40–1:55	**Context managers**
1:55–2:00	**Bring It Home**

Data Science

Advanced OOP in Python



GENERAL ASSEMBLY

Lesson roadmap

Why OOP Matters in Data Science

Inheritance and Overriding

Polymorphism and Method Types

Class vs Instance Attributes

Access Conventions and Properties

Composition and Context Managers

Bring It Home

Learning objectives



1. Use inheritance, overriding, and polymorphism to design reusable Python classes.
2. Distinguish between instance methods, class methods, and static methods.
3. Explain class attributes, instance attributes, and Python access conventions.
4. Use properties to validate object state.
5. Use composition to connect workflow components.
6. Use context managers for predictable setup and cleanup.

Scenario

You are a data scientist at FreshCart, an online grocery delivery company.

FreshCart wants to predict whether an order will arrive late so customer-support teams can intervene before a delivery issue occurs.

The workflow uses:

- CSV files containing historical orders
- API data containing live courier updates
- Experiment configs for different models
- Experiment tracking for model results
- Safe resource handling for files and connections



We will use OOP to make these components reusable, testable, and easier to extend.

Inheritance, Overriding, and Polymorphism

Why Advanced OOP Matters

In data science, OOP helps us build reusable components for:

- Data loading
- Data validation
- Model configuration
- Feature pipelines
- Experiment tracking
- Workflow orchestration

Good OOP design helps us avoid writing the same logic again and again

Moving From General to Specific Classes

Nearly every application will have the concept of a **User**. However, there are more specific types of users with different functionality and properties:

- **Admin** users have access to nearly everything behind the scenes
- **Moderators** might have editing capabilities over user-created content
- **Team leaders** have authority over a specific group of user accounts

Depending on the application, a wide range of more specific **Users** could be necessary to restrict or enhance a given user's abilities.

Creating Specific Sub-Classes with Inheritance

One of the most powerful concepts of Object-Oriented Programming is the ability for a **class to inherit base functionality from a parent class**, then extend in more specific ways.

The **User** class in our example would contain everything common to all users.

The **Admin** class would have all the same functionality as a regular **User**, plus more specific properties and methods.

Inheritance Example

```
class User:
    all_users = []
    can_post = {}

    def __init__(self, username):
       self.username = username
       self.all_users.append(username)
       self.can_post[username] = True
```

```
class Admin(User):
     def __init__(self, username):
          super().__init__(username)

     def suspend_user(username):
       for user in User.all_users:
             if user == username:
                 User.can_post[user] = False
```

Note the `super()` method. It accesses the parent class. Why do we want to call the parent's **`__init__`** function?

Applying this to FreshCart: Inheritance

Inheritance lets one class reuse or extend behaviour from another class.

So a `CSVSource` is a `DataSource`.

Use inheritance when the child class is a specialised version of the parent.

```
class DataSource:
    def load(self):
        pass

class CSVSource(DataSource):
    def load(self):
        ...
```

Overriding

A child class can replace behaviour inherited from a parent class.

This is called **method overriding**.

The method name stays the same, but the child class provides the specific behaviour.

```
class DataSource:
    def load(self):
        raise NotImplementedError

class CSVSource(DataSource):
    def load(self):
        return "Loading CSV data"
```

Polymorphism

Polymorphism means that **different objects can respond to the same method call**.

The loop does not need to know what kind of source each object is.

It only needs to trust that each object has a `.load()` method.

```
sources = [CSVSource(), APISource()]

for source in sources:
    data = source.load()
```

DISCUSSIONS: Inheritance or Not?

Inheritance or Not?

Would you use inheritance here?

1. `CSVSource` and `APISource` both load data.
2. `ModelConfig` stores model parameters.
3. `ExperimentRun` stores a score and timestamp.
4. `LateDeliveryModel` and `FraudModel` both make predictions.
5. `ReportGenerator` uses a trained model to create a PDF.

Be ready to explain your reasoning.

Data Sources

Let's build a small data-source hierarchy.

We will:

1. Create a `DataSource` parent class.
2. Create `CSVSource` and `APISource` child classes.
3. Override `.load()` in each child class.
4. Put different sources into one list.
5. Loop through them using polymorphism.

Open the **02-advanced-oop-in-python.ipynb** notebook and follow along.

Method Types and Class Attributes

Python classes can contain different kinds of methods

Method Type	First Argument	Common Use
Instance	`self`	Uses or changes one object
Class	`cls`	Works with the class itself
Static	none	Utility function stored in the class

The method type should match the job the method is doing

Class vs Instance Attributes

INSTANCE ATTRIBUTES	CLASS ATTRIBUTES
Instance attributes belong to one object.	Class attributes belong to the class and are shared.
`run.score = 0.84`	`ExperimentRun.run_count = 0`
Use class attributes for shared values or counters.	Use instance attributes for object-specific state.

Experiment Tracker

Let's build an `ExperimentRun` class. We will add:

1. Instance attributes for model name and score
2. A class attribute to count runs
3. A static method for score validation
4. A class method to build an object from a dictionary
5. A readable `__repr__`

Open the **02-advanced-oop-in-python.ipynb** notebook and follow along.

Python Access Conventions, Properties, and Composition

Python Access Conventions

Python does not use strict private/protected access like some other languages.

Important

Double underscores ***do not*** make something secure.

Common conventions

```
name        # public
_name       # internal by convention
__name      # name-mangled
```

Properties

Properties let us control how attributes are read and updated.

Use properties when you need validation but want clean attribute-style syntax.

```
@property
def late_threshold(self):
    return self._late_threshold

@late_threshold.setter
def late_threshold(self, value):
    if not 0 <= value <= 1:
        raise ValueError
    self._late_threshold = value
```

 © 2025 General Assembly

Composition

Composition means one object contains or uses another object.

- A workflow **has a** data source.
- A CSV source **is a** data source.

Use composition for **has-a** relationships.

```
workflow = LateDeliveryWorkflow(
    data_source=CSVSource(...),
    config=ModelConfig(...)
)
```

FreshCart Workflow

Let's compose a small late-delivery workflow. We will:

1. Load historical order records.
2. Store model configuration values.
3. Compose a workflow from a data source and config.
4. Generate simple late-delivery predictions.

Open the **02-advanced-oop-in-python.ipynb** notebook and follow along.

Context Managers



Context Managers

Some resources need setup and cleanup:

- Files
- Database connections
- API sessions
- Timers
- Cloud resources

```
with WorkflowTimer("late delivery workflow"):
    workflow.run()
```

Context managers give us predictable setup and cleanup

SKILLS CHALLENGE: Build a Managed Resource

Now You Try: Build a Managed Resource

Before you do this we will have a quick Guided walkthrough in the notebook

Create `DataConnection` as a context manager.

It should:

- ☐ Print `Opening connection` in `__enter__`
- ☐ Print `Closing connection` in `__exit__`
- ☐ Return itself from `__enter__`
- ☐ Have a `.query()` method that returns sample delivery records

Complete it within the relevant space in the **02-advanced-oop-in-python.ipynb** notebook.

PARTNER EXERCISE: Design Check

If Time

Design Check

With a partner, discuss the FreshCart workflow.

For each component, decide whether you would use inheritance, composition, or neither. Components:

- `CSVSource`
- `APISource`
- `ModelConfig`
- `ExperimentRun`
- `LateDeliveryWorkflow`
- `DataConnection`

Be ready to justify one design choice.

Bring It Home

Key takeaways



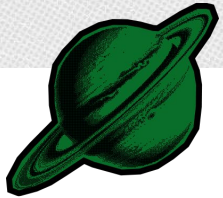
Inheritance and polymorphism support reuse

Specialised classes can share a common interface while providing their own behaviour.



Different class features solve different design problems

Method types, class attributes, properties and access conventions help organise and validate object behaviour.



Composition and context managers improve maintainability

Composition keeps workflows flexible, while context managers provide predictable resource cleanup.

Where next?

If you want to dig deeper into OOP in Python here are some topics to explore next:

- **Abstract base classes:** enforce required methods in child classes.
- **Mixins:** add small, reusable behaviours through inheritance.
- **MRO:** the order Python follows when resolving methods.
- **Destructors:** `__del__()` exists, but context managers are usually safer.

These are included at the end of the notebook as optional stretch content.


GA
GENERAL ASSEMBLY